

Graph Library: Graph Container Interface

Document #: **P3130r1**
Date: 2024-02-05
Project: Programming Language C++
Audience: Library Evolution
SG19 Machine Learning
SG14 Game, Embedded, Low Latency
SG6 Numerics
Revises: P3130r0

Reply-to: Phil Ratzloff (SAS Institute)
phil.ratzloff@sas.com
Andrew Lumsdaine
lumsdaine@gmail.com

Contributors: Kevin Deweese
Muhammad Osama (AMD, Inc)
Jesun Firoz
Michael Wong (Intel)
Jens Maurer
Richard Dosselmann (University of Regina)
Matthew Galati (Amazon)
Guy Davidson (Creative Assembly)

1 Getting Started

This paper is one of several interrelated papers for a proposed Graph Library for the Standard C++ Library. The Table 1 describes all the related papers.

Paper	Status	Description
P1709	Inactive	Original proposal, now separated into the following papers.
P3126	Active	Overview , describes the big picture of what we are proposing.
P3127	Active	Background and Terminology provides the motivation, theoretical background, and terminology used across the other documents.
P3128	Active	Algorithms covers the initial algorithms as well as the ones we'd like to see in the future.
P3129	Active	Views has helpful views for traversing a graph.
P3130	Active	Graph Container Interface is the core interface used for uniformly accessing graph data structures by views and algorithms. It is also designed to easily adapt to existing graph data structures.
P3131	Active	Graph Containers describes a proposed high-performance <code>compressed_graph</code> container. It also discusses how to use containers in the standard library to define a graph, and how to adapt existing graph data structures.

Table 1: Graph Library Papers

Reading them in order will give the best overall picture. If you're limited on time, you can use the following guide to focus on the papers that are most relevant to your needs.

Reading Guide

- If you're **new to the Graph Library**, we recommend starting with the *Overview* paper ([P3126](#)) to understand the focus and scope of our proposals.
- If you want to **understand the theoretical background** that underpins what we're doing, you should read the *Background and Terminology* paper ([P3127](#)).
- If you want to **use the algorithms**, you should read the *Algorithms* paper ([P3128](#)) and *Graph Containers* paper ([P3131](#)).
- If you want to **write new algorithms**, you should read the *Views* paper ([P3129](#)), *Graph Container Interface* paper ([P3130](#)), and *Graph Containers* paper ([P3131](#)). You'll also want to review existing implementations in the reference library for examples of how to write the algorithms.
- If you want to **use your own graph container**, you should read the *Graph Container Interface* paper ([P3130](#)) and *Graph Containers* paper ([P3131](#)).

2 Revision History

P3130r0

- Split from P1709r5. Added *Getting Started* section.
- Add default implementation for `target_id(g,uv)` when the graph type matches the pattern `random_access_range<forward_range<integral>> or random_access_range<forward_range<tuple<integral, ...>>> ; vertex_id_t<G>` also defaults to the `integral` type given.
- Revised concept definitions, adding `sourced_targeted_edge` and `target_edge_range`, and replaced summary table with code for clarity. Also assured that all combinations of adjacency list concepts for *basic*, *sourced* and *index* exist.
- Move text for graph data structures created from std containers from Graph Container Interface to Container Implementation paper.

- Identify all **concept** definitions as "For exposition only" until we have consensus of whether they belong in the standard or not.

P3130r1

- Add `num_edges(g)` and `has_edge(g)` functions. Split function table into 3 tables for graph, vertex and edge functions because it was getting too big.
- Removed the Load Graph Data section with it's load functions from [P3130 Graph Container Interface](#) because it unnecessarily complicates the interface with constructors for graph data structures. To complement this, constructors have been added for `compressed_graph` in [P3131 Graph Containers](#).
- Revised partition functions after implementation in `compressed_graph` to reflect usage, including: renaming `partition_count(g)` to `num_partitions(g)` to match other names used, changed `partition_id(g,u)` to `partition_id(g,uid)` because vertices may not exist when the function is called, and removing `edges(g,u,pid)` because it can easily be implemented as a filter using ranges functionality when target vertices can be in different partitions.

3 Naming Conventions

Table 2 shows the naming conventions used throughout the Graph Library documents.

Template Parameter	Type Alias	Variable Names	Description
G			Graph
	<code>graph_reference_t<G></code>	<code>g</code>	Graph reference
GV		<code>val</code>	Graph Value, value or reference
V	<code>vertex_t<G></code>		Vertex
	<code>vertex_reference_t<G></code>	<code>u,v,x,y</code>	Vertex reference. <code>u</code> is the source (or only) vertex. <code>v</code> is the target vertex.
VId	<code>vertex_id_t<G></code>	<code>uid,vid,seed</code>	Vertex id. <code>uid</code> is the source (or only) vertex id. <code>vid</code> is the target vertex id.
VV	<code>vertex_value_t<G></code>	<code>val</code>	Vertex Value, value or reference. This can be either the user-defined value on a vertex, or a value returned by a function object (e.g. <code>VVF</code>) that is related to the vertex.
VR	<code>vertex_range_t<G></code>	<code>ur,vr</code>	Vertex Range
VI	<code>vertex_iterator_t<G></code>	<code>ui,vi</code>	Vertex Iterator. <code>ui</code> is the source (or only) vertex.
		<code>first,last</code>	<code>vi</code> is the target vertex.
VVF		<code>vvf</code>	Vertex Value Function: <code>vvf(u) → vertex value</code> , or <code>vvf(uid) → vertex value</code> , depending on requirements of the consume algorithm or view.
VProj		<code>vproj</code>	Vertex descriptor projection function: <code>vproj(x) → vertex_descriptor<VId,VV></code> .
	<code>partition_id_t<G></code>	<code>pid</code>	Partition id.
		<code>P</code>	Number of partitions.
PVR	<code>partition_vertex_range_t<G></code>	<code>pur,pvr</code>	Partition vertex range.
E	<code>edge_t<G></code>		Edge
	<code>edge_reference_t<G></code>	<code>uv,vw</code>	Edge reference. <code>uv</code> is an edge from vertices <code>u</code> to <code>v</code> . <code>vw</code> is an edge from vertices <code>v</code> to <code>w</code> .
EId	<code>edge_id_t<G></code>	<code>eid,uvid</code>	Edge id, a pair of vertex_ids.
EV	<code>edge_value_t<G></code>	<code>val</code>	Edge Value, value or reference. This can be either the user-defined value on an edge, or a value returned by a function object (e.g. <code>EVF</code>) that is related to the edge.
ER	<code>vertex_edge_range_t<G></code>		Edge Range for edges of a vertex
EI	<code>vertex_edge_iterator_t<G></code>	<code>uvi,vwi</code>	Edge Iterator for an edge of a vertex. <code>uvi</code> is an iterator for an edge from vertices <code>u</code> to <code>v</code> . <code>vwi</code> is an iterator for an edge from vertices <code>v</code> to <code>w</code> .
EVF		<code>evf</code>	Edge Value Function: <code>evf(uv) → edge value</code> , or <code>evf(eid) → edge value</code> , depending on the requirements of the consuming algorithm or view.
EProj		<code>eproj</code>	Edge descriptor projection function: <code>eproj(x) → edge_descriptor<VId,Sourced,EV></code> .

Table 2: Naming Conventions for Types and Variables

4 Graph Container Interface

The Graph Container Interface defines the primitive concepts, traits, types and functions used to define and access an adjacency graph, no matter its internal design and organization. Thus, it is designed to reflect all forms of adjacency graphs including a vector of lists, CSR-based graph and adjacency matrix, whether they are in the standard or external to the standard.

All algorithms in this proposal require that vertices are stored in random access containers and that `vertex_id_t<G>` is integral, and it is assumed that all future algorithm proposals will also have the same requirements.

The Graph Container Interface is designed to support a wider scope of graph containers than required by the views and algorithms in this proposal. This enables for future growth of the graph data model (e.g. incoming edges on a vertex), or as a framework for graph implementations outside of the standard. For instance, existing implementations may have requirements that cause them to define features with looser constraints, such as sparse `vertex_ids`, non-integral `vertex_ids`, or storing vertices in associative bi-directional containers (e.g. `std::map` or `std::unordered_map`). Such features require specialized implementations for views and algorithms. The performance for such algorithms will be sub-optimal, but is preferable to run them on the existing container rather than loading the graph into a high-performance graph container and then running the algorithm on it, where the loading time can far outweigh the time to run the sub-optimal algorithm. To achieve this, care has been taken to make sure that the use of concepts chosen is appropriate for algorithm, view and container.

5 Core Concepts

This section describes the core concepts to describe the adjacency lists used for graphs in the Graph Library. There are a number of qualifiers concept names that are used.

- **basic** where there is only a vertex id but no vertex reference.
- **index** where the vertex range is random-access and the vertex id is integral.
- **sourced** where an edge has a source id.

While we believe the use of concepts is appropriate for graphs as a range-of-ranges, we are marking them as "For exposition only" until we have consensus of whether they belong in the standard or not.

5.1 Edge Concepts

The types of edges that can occur in a graph are described with the edges concepts. The `E` edge template parameter allows for different types of edges, such as incoming and outgoing edge types.

```
// For exposition only

template <class G, class E>
concept basic_targeted_edge = requires(G&& g, edge_reference_t<G> uv) { target_id(g, uv); };

template <class G, class E>
concept basic_sourced_edge = requires(G&& g, edge_reference_t<G> uv) { source_id(g, uv); };

template <class G, class E>
concept basic_sourced_targeted_edge = basic_targeted_edge<G, E> && //
                                     basic_sourced_edge<G, E> && //
                                     requires(G&& g, edge_reference_t<G> uv) { edge_id(g, uv); };

template <class G, class E>
concept targeted_edge = basic_targeted_edge<G, E> && //
                       requires(G&& g, edge_reference_t<G> uv) { target(g, uv); };

template <class G, class E>
concept sourced_edge = basic_sourced_edge<G, E> && //
                      requires(G&& g, edge_reference_t<G> uv) { source(g, uv); };
```

```
template <class G, class E>
concept sourced_targeted_edge = targeted_edge<G, E> && //
    sourced_edge<G, E> && //
    requires(G&& g, edge_reference_t<G> uv) { edge_id(g, uv); };
```

There are two edge range concepts.

[PHIL: `edges(g,u)` isn't being used anywhere. We could use `basic_adjacency_list` in the View and algorithm definitions.]

```
// For exposition only

template <class G>
concept basic_targeted_edge_range = requires(G&& g, vertex_id_t<G> uid) {
    { edges(g, uid) } -> ranges::forward_range;
};

template <class G>
concept targeted_edge_range = basic_targeted_edge_range<G> && //
    requires(G&& g, vertex_reference_t<G> u) {
        { edges(g, u) } -> ranges::forward_range;
    };
};
```

5.2 Vertex Concepts

The `vertex_range` concept is the general definition used for adjacency lists while `index_vertex_range` is used for high performance graphs where vertices typically stored in a `vector`.

```
// For exposition only

template <class G> // (exposition only)
concept _common_vertex_range = ranges::sized_range<vertex_range_t<G>> && //
    requires(G&& g, vertex_iterator_t<G> ui) { vertex_id(g, ui); };

template <class G>
concept vertex_range = _common_vertex_range<vertex_range_t<G>> && //
    ranges::forward_range<vertex_range_t<G>>;

template <class G>
concept index_vertex_range = _common_vertex_range<vertex_range_t<G>> && //
    ranges::random_access_range<vertex_range_t<G>> && //
    integral<vertex_id_t<G>>;
```

5.3 Adjacency List Concepts

The basic adjacency lists add concepts where there is no vertex object, only a vertex id.

```
// For exposition only

template <class G>
concept basic_adjacency_list = vertex_range<G> && //
    basic_targeted_edge_range<G> && //
    targeted_edge<G, edge_t<G>>;

template <class G>
concept basic_index_adjacency_list = index_vertex_range<G> && //
    basic_targeted_edge_range<G> && //
```

```

                                basic_targeted_edge<G, edge_t<G>>;
template <class G>
concept basic_sourced_adjacency_list = vertex_range<G> && //
                                basic_targeted_edge_range<G> && //
                                basic_sourced_targeted_edge<G, edge_t<G>>;

template <class G>
concept basic_sourced_index_adjacency_list = index_vertex_range<G> && //
                                basic_targeted_edge_range<G> && //
                                basic_sourced_targeted_edge<G, edge_t<G>>;

```

The adjacency list concepts bring together the vertex and edge concepts used for core graph concepts. All algorithms initially proposed for the Graph Library use the `index_adjacency_list`.

```

// For exposition only

template <class G>
concept adjacency_list = vertex_range<G> && //
                        targeted_edge_range<G> && //
                        targeted_edge<G, edge_t<G>>;

template <class G>
concept index_adjacency_list = index_vertex_range<G> && //
                        targeted_edge_range<G> && //
                        targeted_edge<G, edge_t<G>>;

template <class G>
concept sourced_adjacency_list = vertex_range<G> && //
                        targeted_edge_range<G> && //
                        sourced_targeted_edge<G, edge_t<G>>;

template <class G>
concept sourced_index_adjacency_list = index_vertex_range<G> && //
                        targeted_edge_range<G> && //
                        sourced_targeted_edge<G, edge_t<G>>;

```

6 Traits

Table 3 summarizes the type traits in the Graph Container Interface, allowing views and algorithms to query the graph's characteristics.

7 Types

Table 4 summarizes the type aliases in the Graph Container Interface. These are the types used to define the objects in a graph container, no matter its internal design and organization. Thus, it is designed to be able to reflect all forms of adjacency graphs including a vector of lists, `compressed_graph` and adjacency matrix.

The type aliases are defined by either a function specialization for the underlying graph container, or a refinement of one of those types (e.g. an iterator of a range). Table 5 describes the functions in more detail.

`graph_value(g)`, `vertex_value(g,u)` and `edge_value(g,uv)` can be optionally implemented, depending on whether the graph container supports values on the graph, vertex and edge types.

There is no contiguous requirement for `vertex_id` from one partition to the next, though in practice they will often be assigned contiguously. Gaps in `vertex_id`s between partitions should be allowed.

Trait	Type	Comment
<code>has_degree<G></code>	concept	Is the <code>degree(g,u)</code> function available?
<code>has_find_vertex<G></code>	concept	Are the <code>find_vertex(g,_)</code> functions available?
<code>has_find_vertex_edge<G></code>	concept	Are the <code>find_vertex_edge(g,_)</code> functions available?
<code>has_contains_edge<G></code>	concept	Is the <code>contains_edge(g,uid,vid)</code> function available?
<code>define_unordered_edge<G,E> : false_type</code>	struct	Specialize for edge implementation to derive from <code>true_type</code> for unordered edges
<code>is_unordered_edge<G,E></code>	struct	<code>conjunction<define_unordered_edge<E>, is_sourced_edge<G, E>></code>
<code>is_unordered_edge_v<G,E></code>	type alias	
<code>unordered_edge<G,E></code>	concept	
<code>is_ordered_edge<G,E></code>	struct	<code>negation<is_unordered_edge<G,E>></code>
<code>is_ordered_edge_v<G,E></code>	type alias	
<code>ordered_edge<G,E></code>	concept	
<code>define_adjacency_matrix<G> : false_type</code>	struct	Specialize for graph implementation to derive from <code>true_type</code> for edges stored as a square 2-dimensional array
<code>is_adjacency_matrix<G></code>	struct	
<code>is_adjacency_matrix_v<G></code>	type alias	
<code>adjacency_matrix<G></code>	concept	

Table 3: Graph Container Interface Type Traits

Type Alias	Definition	Comment
<code>graph_reference_t<G></code>	<code>add_lvalue_reference<G></code>	
<code>graph_value_t<G></code>	<code>decltype(graph_value(g))</code>	optional
<code>vertex_range_t<G></code>	<code>decltype(vertices(g))</code>	
<code>vertex_iterator_t<G></code>	<code>iterator_t<vertex_range_t<G>></code>	
<code>vertex_t<G></code>	<code>range_value_t<vertex_range_t<G>></code>	
<code>vertex_reference_t<G></code>	<code>range_reference_t<vertex_range_t<G>></code>	
<code>vertex_id_t<G></code>	<code>decltype(vertex_id(g,ui))</code>	
<code>vertex_value_t<G></code>	<code>decltype(vertex_value(g,u))</code>	optional
<code>vertex_edge_range_t<G></code>	<code>decltype(edges(g,u))</code>	
<code>vertex_edge_iterator_t<G></code>	<code>iterator_t<vertex_edge_range_t<G>></code>	
<code>edge_t<G></code>	<code>range_value_t<vertex_edge_range_t<G>></code>	
<code>edge_reference_t<G></code>	<code>range_reference_t<vertex_edge_range_t<G>></code>	
<code>edge_value_t<G></code>	<code>decltype(edge_value(g,uv))</code>	optional
The following is only available when the optional <code>source_id(g,uv)</code> is defined for the edge		
<code>edge_id_t<G></code>	<code>edge_descriptor<vertex_id_t<G>, true, void, void></code>	
<code>partition_id_t<G></code>	<code>decltype(partition_id(g,uid))</code>	optional
<code>partition_vertex_range_t<G></code>	<code>vertices(g,pid)</code>	optional

Table 4: Graph Container Interface Type Aliases

8 Classes and Structs

The `graph_error` exception class is available, inherited from `runtime_error`. While any function may use it, it is only anticipated to be used by the `load` functions at this time. No additional functionality is added beyond that

provided by `runtime_error`.

While we believe the use of concepts is appropriate for graphs as a range-of-ranges, we are marking them as "For exposition only" until we have consensus of whether they belong in the standard or not.

9 Functions

Tables 5, 6 and 7 summarize the primitive functions in the Graph Container Interface. used to access an adjacency graph, no matter its internal design and organization. Thus, it is designed to be able to reflect all forms of adjacency graphs including a vector of lists, CSR-based graph and adjacency matrix.

Function	Return Type	Complexity	Default Implementation
<code>graph_value(g)</code>	<code>graph_value_t<G></code>	constant	n/a, optional
<code>vertices(g)</code>	<code>vertex_range_t<G></code>	constant	<code>g</code> if <code>random_access_range<G></code> , n/a otherwise
<code>num_vertices(g)</code>	<code>integral</code>	constant	<code>size(vertices(g))</code>
<code>num_edges(g)</code>	<code>integral</code>	$ E $	<code>n=0; for(u: vertices(g)) n+=distance(edges(g,u)); return n;</code>
<code>has_edge(g)</code>	<code>bool</code>	$ V $	<code>for(u: vertices(g)) if !empty(edges(g,u)) return true; return false;</code>
<code>num_partitions(g)</code>	<code>integral</code>	constant	1
<code>vertices(g,pid)</code>	<code>partition_vertex_range_t<G></code>	constant	<code>vertices(g)</code>
<code>num_vertices(g,pid)</code>	<code>integral</code>	constant	<code>size(vertices(g))</code>

Table 5: Graph Functions

The complexity shown above for `num_edges(g)` and `has_edge(g)` is for the default implementation. Specific graph implementations may have better characteristics.

The complexity shown above for `vertices(g,pid)` and `num_vertices(g,pid)` is for the default implementation. Specific graph implementations may have different characteristics.

Function	Return Type	Complexity	Default Implementation
<code>find_vertex(g,uid)</code>	<code>vertex_iterator_t<G></code>	constant	<code>begin(vertices(g))+ uid</code>
<code>vertex_id(g,ui)</code>	<code>vertex_id_t<G></code>	constant	if <code>random_access_range<vertex_range_t<G>></code> <code>(size_t)(ui - begin(vertices(g)))</code> Override to define a different <code>vertex_id_t<G></code> type (e.g. <code>int32_t</code>).
<code>vertex_value(g,u)</code>	<code>vertex_value_t<G></code>	constant	n/a, optional
<code>vertex_value(g,uid)</code>	<code>vertex_value_t<G></code>	constant	<code>vertex_value(g,*find_vertex(g,uid))</code> , optional
<code>degree(g,u)</code>	<code>integral</code>	constant	<code>size(edges(g,u))</code> if <code>sized_range<vertex_edge_range_t<G>></code>
<code>degree(g,uid)</code>	<code>integral</code>	constant	<code>size(edges(g,uid))</code> if <code>sized_range<vertex_edge_range_t<G>></code>
<code>edges(g,u)</code>	<code>vertex_edge_range_t<G></code>	constant	<code>u</code> if <code>forward_range<vertex_t<G>></code> , n/a otherwise
<code>edges(g,uid)</code>	<code>vertex_edge_range_t<G></code>	constant	<code>edges(g,*find_vertex(g,uid))</code>
<code>partition_id(g,uid)</code>	<code>partition_id_t<G></code>	constant	0

Table 6: Vertex Functions

The default implementation for the `degree` functions assumes that `vertex_edge_range_t<G>` is a sized range to have constant complexity. If the underlying container has a non-linear `size(R)` function, the `degree` functions will also be non-linear. This is expected to be an uncommon case.

Function	Return Type	Complexity	Default Implementation
<code>target_id(g,uv)</code>	<code>vertex_id_t<G></code>	constant	(see below)
<code>target(g,uv)</code>	<code>vertex_t<G></code>	constant	<code>*(begin(vertices(g))+ target_id(g, uv))</code> if <code>random_access_range<vertex_range_t<G>> && integral<target_id(g,uv)></code>
<code>edge_value(g,uv)</code>	<code>edge_value_t<G></code>	constant	<code>uv</code> if <code>forward_range<vertex_t<G>></code> , n/a otherwise, optional
<code>find_vertex_edge(g,u,vid)</code>	<code>vertex_edge_t<G></code>	linear	<code>find(edges(g,u), [] (uv)target_id(g,uv)==vid;}}</code>
<code>find_vertex_edge(g,uid,vid)</code>	<code>vertex_edge_t<G></code>	linear	<code>find_vertex_edge(g,*find_vertex(g,uid),vid)</code>
<code>contains_edge(g,uid,vid)</code>	<code>bool</code>	constant	<code>uid < size(vertices(g))&& vid < size(vertices(g))</code> if <code>is_adjacency_matrix_v<G></code> .
		linear	<code>find_vertex_edge(g,uid)!= end(edges(g,uid))</code> otherwise.
----- The following are only available when the optional <code>source_id(g,uv)</code> is defined for the edge -----			
<code>source_id(g,uv)</code>	<code>vertex_id_t<G></code>	constant	n/a, optional
<code>source(g,uv)</code>	<code>vertex_t<G></code>	constant	<code>*(begin(vertices(g))+ source_id(g,uv))</code> if <code>random_access_range<vertex_range_t<G>> && integral<target_id(g,uv)></code>
<code>edge_id(g,uv)</code>	<code>edge_id_t<G></code>	constant	<code>edge_descriptor<vertex_id_t<G>,true,void, void>{source_id(g,uv),target_id(g,uv)}</code>

Table 7: Edge Functions

When the graph matches the pattern `random_access_range<forward_range<integral>>` or `random_access_range<forward_range<tuple<integral,...>>>` , the default implementation for `target_id(g,uv)` will return the `integral` . Additionally, if the caller does not override `vertex_id(g,ui)` , the `integral` value will define the `vertex_id_t<G>` type.

Functions that have n/a for their Default Implementation must be defined by the author of a Graph Container implementation.

Value functions (`graph_value(g)` , `vertex_value(g,u)` and `edge_value(g,uv)`) can be optionally implemented, depending on whether the graph container supports values on the graph, vertex and edge types. They return a single value and can be scalar, struct, class, union, or tuple. These are abstract types used by the GVF, VVF and EVF function objects to retrieve values used by algorithms. As such it's valid to return the "enclosing" owning class (graph, vertex or edge), or some other embedded value in those objects.

`find_vertex(g,uid)` is constant complexity because all algorithms in this proposal require that `vertex_range_t<G>` is a random access range.

If the concept requirements for the default implementation aren't met by the graph container the function will need to be overridden.

9.1 Determining the Vertex Id Type

To determine the type for `vertex_id_t<G>` the following steps are taken, in order, to determine its type.

1. Use the type returned by `vertex_id(g,ui)` when overridden for a graph.
2. When the graph matches the pattern `random_access_range<forward_range<integral>>` or `random_access_range<forward_range<tuple<integral,...>>>` , use the `integral` type specified.
3. Use `size_t` in all other cases.

`vertex_id_t<G>` is defined by the type returned by `vertex_id(g)` and it defaults to the `difference_type` of the underlying container used for vertices (e.g `int64_t` for 64-bit systems). This is sufficient for all situations.

However, there are often space and performance advantages if a smaller type is used, such as `int32_t` or even `int16_t`. It is recommended to consider overriding this function for optimal results, assuring that it is also large enough for the number of possible vertices and edges in the application. It will also need to be overridden if the implementation doesn't expose the vertices as a range.

10 Unipartite, Bipartite and Multipartite Graph Representation

`num_partitions(g)` returns the number of partitions, or partiteness, of the graph. It has a range of 1 to n , where 1 identifies a unipartite graph, 2 is a bipartite graph, and a value of 2 or more can be considered a multipartite graph.

If a graph data structure doesn't support partitions then it is unipartite with one partition and partite functions will reflect that. For instance, `num_partitions(g)` returns a value of 1, and `vertices(g,0)` (vertices in the first partition) will return a range that includes all vertices in the graph.

A partition identifies a type of a vertex, where the vertex value types are assumed to be uniform in each partition. This creates a dilemma because the existing `vertex_value(g,u)` returns a single type based template parameter for the vertex value type. Supporting multiple types can be addressed in different ways using C++ features. The key to remember is that the actual value used by algorithms is done by calling a function object that retrieves the value to be used. That function is specific to the graph data structure, using the partition to determine how to get the appropriate value.

- `std::variant` : The lambda returns the appropriate variant value based on the partition.
- Base class pointer: The lambda can call a member function to return the value based on the partition.
- `void*` : The lambda can cast the pointer to a concrete type based on the partition, and then return the appropriate value.

`edges(g,uid,pid)` and `edges(g,ui,pid)` filter the edges where the target is in the partition `pid` passed. This isn't needed for bipartite graphs.

11 Edgelists

An edgelist is a range of `edge_descriptor` values, used by some algorithms. Examples include the `edgelist` adjacency list view, or possibly a transform of an existing range that projects `edge_descriptor` values. There is no concrete data structure in this proposal like there is for an adjacency list.

12 Using Existing Graph Data Structures

Reasonable defaults have been defined for the GCI functions to minimize the amount of work needed to adapt an existing graph data structure to be used by the views and algorithms.

There are two cases supported. The first is for the use of standard containers to define the graph and the other is for a broader set of more complicated implementations.

This is described in more detail in the paper for Graph Library Container Implementations.

Acknowledgements

Phil Ratzloff's time was made possible by SAS Institute.

Portions of *Andrew Lumsdaine's* time was supported by NSF Award OAC-1716828 and by the Segmented Global Address Space (SGAS) LDRD under the Data Model Convergence (DMC) initiative at the U.S. Department of Energy's Pacific Northwest National Laboratory (PNNL). PNNL is operated by Battelle Memorial Institute under Contract DE-AC06-76RL01830.

Michael Wong's work is made possible by Codeplay Software Ltd., ISOCPP Foundation, Khronos and the Standards Council of Canada.

Muhammad Osama's time was made possible by Advanced Micro Devices, Inc.

The authors thank the members of SG19 and SG14 study groups for their invaluable input.